R3.05 · TD-TP 03

PROGRAMMATION SYSTÈME

# Les pointeurs

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

*Un processus suspect a caché ses secrets dans la mémoire du LAMAE. Ta mission : cartographier les adresses, manipuler les pointeurs comme des passe-partout, déjouer les pièges du labyrinthe et prendre le contrôle du standard téléphonique… par pointeurs de fonctions.*

**OBJECTIFS DE LA SÉANCE**

- ☐ Expliquer la différence entre une variable, son **adresse** (`&`) et la valeur **pointée** (`*`).
- ☐ Distinguer `p1 = p2` (copie d'adresse) et `*p1 = *p2` (copie de valeur).
- ☐ Utiliser l'arithmétique des pointeurs pour parcourir un tableau **sans en sortir**.
- ☐ Passer des adresses à une fonction pour qu'elle modifie les variables de l'appelant (`swap`).
- ☐ Reconnaître et corriger les erreurs mémoire classiques : pointeur non initialisé, `NULL`, pointeur pendant, débordement.
- ☐ Déclarer, ranger dans un tableau et appeler des **pointeurs de fonctions**.

**FIL ROUGE** **Opération « Mémoire vive »**

Le LAMAE a capturé un processus espion. Pour le neutraliser, il faut récupérer **cinq fragments de clé** de la forme `FLAG{...}`, un par mission. Notez-les dans `badge03.txt`. Les fichiers de départ sont dans `code/sujet03/`. Rappel de la règle d'or : on compile avec `gcc -std=c17 -Wall -Wextra -pedantic -g`, et on **lit** les avertissements.

| Mission | Contenu | Durée |
|---|---|---|
| 0 · Le cartographe | Adresses, `&`, `sizeof`, table objet/adresse/valeur | 15 min |
| 1 · Le passe-partout | Pointeurs, `*`, `p1 = p2` contre `*p1 = *p2`, `swap` | 25 min |
| 2 · L'arpenteur | Arithmétique des pointeurs, parcours de tableau | 25 min |
| 3 · Le labyrinthe piégé | Erreurs mémoire, ASan, `const` | 20 min |
| 4 · Le standard | Pointeurs de fonctions, calculatrice à menu | 35 min |
| Bonus | Pour les rapides : `qsort` et fonctions de rappel | — |

## Mission 0 — Le cartographe : adresses et variables

Toute variable d'un programme est rangée quelque part en mémoire centrale. Cette mémoire est une suite d'**octets**, chacun identifié par un numéro : son **adresse**. Pour retrouver une variable, il suffit de connaître l'adresse de l'octet où elle commence (si elle occupe plusieurs octets contigus, c'est l'adresse du premier).

Pour la lisibilité, on désigne les variables par des identificateurs : c'est le compilateur qui fait le lien entre le nom et l'adresse. Mais il est parfois très utile de manipuler une variable **par son adresse**.

```c
int i, j;
i = 3;
j = i;
```

Si le compilateur a placé `i` à l'adresse 4831836000 et `j` à l'adresse 4831836004, on a :

| objet | adresse | valeur |
|---|---|---|
| `i` | 4831836000 | 3 |
| `j` | 4831836004 | 3 |

Deux variables différentes ont des adresses différentes. L'affectation `j = i;` n'opère que sur les **valeurs**. Un `int` occupant ici 4 octets, la valeur de `i` est stockée sur les octets 4831836000 à 4831836003.

> REMARQUE **Des adresses qui changent à chaque exécution**
> Les adresses de ce sujet (4831836000…) sont des exemples. Sur votre machine, elles seront différentes, et **changeront même à chaque exécution** : Linux place la pile, le tas et les bibliothèques à des positions aléatoires (**ASLR**, *Address Space Layout Randomization*), une protection qui complique l'exploitation des débordements de tampon.

L'opérateur `&` donne l'adresse d'une variable. `&i` n'est pas une variable : on ne peut pas écrire `&i = ...`. Pour stocker et manipuler des adresses, il faut un nouveau type d'objet : le **pointeur**.

### EX 1 Dresser la carte ★ 15 min

Compilez et exécutez `adresses.c` (dossier `code/sujet03/`). Il affiche l'adresse de plusieurs variables avec le format `%p`.

1. Relevez les adresses de `i`, `j`, `x`, `c` et des cases de `tab`. De combien d'octets sont-elles espacées ? Comparez avec `sizeof` de chaque type : complétez le `TODO` pour afficher les tailles avec `%zu`.
2. Les cases de `tab` sont-elles contiguës ? Quel est l'écart entre `&tab[0]` et `&tab[1]` ?
3. Exécutez le programme deux fois de suite. Les adresses de `i` et de `globale` ont-elles changé ? Lesquelles changent le plus ? (Pensez à ASLR.)
4. L'adresse de `globale` est-elle proche de celle de `i` ? Dans quelles zones de la mémoire du processus se trouvent-elles (pile, données) ?

**Fragment 1 :** `FLAG{` + l'écart en octets entre `&tab[0]` et `&tab[3]` + `}`.

> PIÈGE CLASSIQUE **Afficher une adresse**
> Une adresse s'affiche avec `%p`, et l'argument doit être un `void *` : `printf("%p\n", (void *)&i);`. Les formats `%d` ou `%ld` sont faux (taille et type incorrects : comportement indéfini, et `-Wall` le signale).

# Mission 1 — Le passe-partout : la notion de pointeur

Un **pointeur** est un objet dont la valeur est l'adresse d'un autre objet. On le déclare ainsi :

```c
type *nomDuPointeur;
```

où `type` est le type de l'objet pointé. Même si un pointeur contient une adresse, son **type dépend de l'objet pointé** : pour un `char *`, l'adresse désigne un octet ; pour un `int *`, elle désigne le premier des 4 octets de l'entier. Cette distinction est indispensable pour interpréter la mémoire pointée (et, on le verra, pour l'arithmétique).

```c
int i = 3;
int *p;      // déclaration d'un pointeur vers un int
p = &i;      // p reçoit l'adresse de i : p pointe sur i
```

| objet | adresse | valeur |
|---|---|---|
| `i` | 4831836000 | 3 |
| `p` | 4831836008 | 4831836000 |

L'opérateur unaire d'**indirection** `*` donne accès à l'objet pointé : si `p` pointe sur `i`, `*p` **est** `i`.

```c
#include <stdio.h>

int main(void)
{
    int i = 3;
    int *p = &i;               // int * : pointeur vers un int
    printf("*p = %d\n", *p);   // *p : valeur pointée par p -> 3
    *p = 0;                    // modifie i !
    printf("i = %d\n", i);     // -> 0
    return 0;
}
```

Ici `i` et `*p` sont **le même objet** : même adresse, même valeur. Toute modification de `*p` modifie `i`.

| objet | adresse | valeur |
|---|---|---|
| `i` | 4831836000 | 3 |
| `p` | 4831836008 | 4831836000 |
| `*p` | 4831836000 | 3 |

## p1 = p2 ou *p1 = *p2 ?

On manipule donc à la fois `p` (une adresse) et `*p` (la valeur pointée). Comparez ces deux programmes :

```c
int i = 3, j = 6;
int *p1 = &i;
int *p2 = &j;
*p1 = *p2;     // copie la VALEUR pointée
```

```c
int i = 3, j = 6;
int *p1 = &i;
int *p2 = &j;
p1 = p2;       // copie l'ADRESSE
```

Avant la dernière affectation, on est dans la configuration :

| objet | adresse | valeur |
|---|---|---|
| `i` | 4831836000 | 3 |

| objet | adresse | valeur |
|---|---|---|
| `j` | 4831836004 | 6 |
| `p1` | 4831835984 | 4831836000 |
| `p2` | 4831835992 | 4831836004 |

### EX 2 Prédire avant d'exécuter ★ 10 min

Sans exécuter, complétez sur papier la table objet/adresse/valeur **après** la dernière affectation de chacun des deux programmes. Vérifiez ensuite en ajoutant des `printf` (adresses en `%p`).

1. Dans quel programme `i` a-t-il changé ?
2. Dans le programme B, si l'on écrit ensuite `*p1 = 10;`, quelle variable est modifiée ?
3. Mini-quiz : que vaut `i` à la fin de la séquence suivante ?

```c
int i = 1, j = 2, k = 3;
int *p = &i, *q = &j;
*p = *q + k;      // ?
p = q;            // ?
*p = *p * 10;     // ?
q = &k;
*q = *p + i;      // ?
```

**Fragment 2 :** `FLAG{` + valeur finale de `i` + `-` + valeur finale de `k` + `}`.

### EX 3 La fonction swap ★ 15 min

Programmez une fonction nommée `swap` dont le rôle est d'**échanger** la valeur de deux variables de type `int` : la valeur de `a` doit devenir celle de `b` et réciproquement.

1. Écrivez d'abord `void swapFaux(int a, int b)` qui échange ses paramètres, et constatez que les variables du `main` ne changent pas. Pourquoi ?
2. Écrivez la bonne version : quels paramètres faut-il passer pour qu'une fonction puisse modifier les variables de l'appelant ?
3. Vous connaissez déjà une fonction de la bibliothèque qui reçoit des adresses pour écrire dedans. Laquelle ?

> **INDICE**
>
> En C, les paramètres sont **toujours passés par valeur** : la fonction reçoit une copie. Pour modifier une variable de l'appelant, on lui passe une copie… de son **adresse**. C'est exactement pourquoi on écrit `scanf("%d", &n)`.

## Mission 2 — L'arpenteur : arithmétique des pointeurs

La valeur d'un pointeur est une adresse, sur laquelle on peut faire quelques opérations. Les seules opérations arithmétiques valides sont :

- l'**addition** d'un entier à un pointeur (résultat : un pointeur de même type) ;
- la **soustraction** d'un entier à un pointeur (résultat : un pointeur de même type) ;
- la **différence** de deux pointeurs vers des éléments **du même tableau** (résultat : un entier, de type `ptrdiff_t`).

La somme de deux pointeurs n'a pas de sens et est interdite.

Si `p` est un pointeur sur `type` et `i` un entier, `p + i` désigne l'adresse de `p` **augmentée de** `i * sizeof(type)` **octets** : on avance de `i` éléments, pas de `i` octets. Il en va de même pour la soustraction et pour `++` et `--`.

C

```c
#include <stdio.h>

int main(void)
{
    int tab[2] = {3, 4};
    int *p1 = &tab[0];
    int *p2 = p1 + 1;                       // avance d'UN int, soit 4 octets
    printf("p1 = %p\tp2 = %p\n", (void *)p1, (void *)p2);
    return 0;
}
```

Affichage possible : `p1 = 0x7ffd5c1e3a40  p2 = 0x7ffd5c1e3a44`. Avec des `double`, l'écart serait de 8 octets.

> **PIÈGE CLASSIQUE** **Pointer « un après », oui ; déréférencer, non**
>
> Le programme d'origine faisait `p2 = &i; p1 = p2++;` sur une simple variable `i`. C'est légal **uniquement** parce que la norme autorise un pointeur « juste après » la fin d'un objet (un objet isolé compte comme un tableau d'un élément). On peut le comparer, mais **jamais** lire ou écrire `*p2` : c'est en dehors de l'objet. Toute autre position hors du tableau (y compris **avant** le premier élément) est un comportement indéfini, même sans déréférencement.

Les opérateurs de comparaison s'appliquent aussi aux pointeurs vers un même tableau. L'arithmétique des pointeurs est particulièrement utile pour parcourir un tableau :

parcours.c

```c
#include <stdio.h>

#define N 5

int main(void)
{
    int tab[N] = {1, 2, 6, 0, 7};
    int *p;

    printf("ordre croissant :\n");
    for (p = &tab[0]; p < tab + N; p++)       // tab + N : « un après », jamais lu
        printf(" %d\n", *p);

    printf("ordre décroissant :\n");
    for (p = tab + N; p > tab; ) {            // on part de « un après »...
        p--;                                  // ...et on recule AVANT de lire
        printf(" %d\n", *p);
    }
    return 0;
}
```

Si `p` et `q` pointent dans le même tableau, `p - q` est le **nombre d'éléments** qui les séparent (et non un nombre d'octets) : le compilateur divise déjà par `sizeof(type)`. Ce résultat est de type `ptrdiff_t` (`<stddef.h>`) et s'affiche avec `%td`.

## EX 4 Observer l'arpentage

★ 10 min

1. Reprenez le premier exemple avec des `double` et des `char` à la place des `int` : de combien d'octets avance `p + 1` dans chaque cas ?

2. Affichez `&tab[4] - &tab[1]` avec `%td`, puis `(char *)&tab[4] - (char *)&tab[1]`. Expliquez les deux résultats.
3. Pourquoi la forme `for (p = &tab[N-1]; p >= &tab[0]; p--)`, que l'on rencontre souvent, est-elle incorrecte en C ? (Quelle est la valeur de `p` à la sortie de la boucle ?)

**Fragment 3 :** `FLAG{` + le résultat de `(char *)&tab[4] - (char *)&tab[1]` pour un tableau de `double` + `}`.

### EX 5 — Compter les occurrences ★ 15 min

Écrivez un programme qui, étant donné un tableau d'entiers déjà initialisé, demande à l'utilisateur quel entier chercher (en vérifiant le retour de `scanf`) et affiche le nombre d'occurrences de cet entier dans le tableau.

- Écrivez la recherche dans une fonction `size_t compterOccurrences(const int *tab, size_t n, int valeur)` qui parcourt le tableau **avec un pointeur** (pas d'indice `[k]`).
- Pourquoi une fonction qui reçoit un tableau a-t-elle **toujours** besoin de sa taille `n` en paramètre ? (Que vaut `sizeof tab` dans la fonction ?)

### EX 6 — Deux pointeurs face à face ★★ 10 min

1. Écrivez `void inverser(int *debut, int *fin)` qui inverse le tableau situé entre `debut` (inclus) et `fin` (exclu, « un après ») en faisant se rapprocher deux pointeurs, en réutilisant votre `swap`.
2. Écrivez `size_t maStrlen(const char *s)` qui calcule la longueur d'une chaîne en avançant un pointeur jusqu'au `'\0'`, puis renvoie une **différence de pointeurs**. Comparez avec `strlen`.

## Mission 3 — Le labyrinthe piégé : programmer sûr avec les pointeurs

Un pointeur mal maîtrisé est la porte d'entrée de la majorité des failles des programmes C. Le fichier `pieges.c` contient **quatre erreurs mémoire volontaires**… et il compile **sans aucun avertissement**.

### EX 7 — Chasse aux pièges ★★ 15 min

1. Compilez `pieges.c` normalement et exécutez-le sans argument, puis avec l'argument `2`. Que se passe-t-il ? Le résultat est-il toujours le même ?
2. Recompilez avec `-fsanitize=address,undefined -g` et exécutez. Lisez le rapport : type d'erreur, ligne fautive. Corrigez, recompilez, et recommencez jusqu'à ce que le programme soit propre.
3. Pour chacun des 4 pièges, nommez l'erreur dans le tableau ci-dessous et indiquez votre correction.
4. Relancez avec l'argument `1000`, puis `-3`. Quelle vérification manquait ?

| Piège | Nom de l'erreur | Correction proposée |
|---|---|---|
| A | | |
| B | | |
| C | | |
| D | | |

**SÉCURITÉ** **Les quatre cavaliers de l'apocalypse mémoire**

- **Pointeur non initialisé** : une variable locale non initialisée contient n'importe quoi. Un pointeur non initialisé pointe donc n'importe où. Initialisez **toujours** un pointeur, au besoin à `NULL`.
- **Déréférencement de NULL** : une fonction qui peut échouer renvoie souvent `NULL` (`malloc`, `fopen`, `strchr`…). Testez **avant** d'utiliser `*p`.
- **Pointeur pendant** (*dangling pointer*) : une variable locale disparaît à la fin de sa fonction. Renvoyer son adresse donne un pointeur vers une zone de pile réutilisée ensuite par d'autres appels.
- **Débordement par arithmétique** : `p <= tab + N` lit une case de trop ; un indice venant de l'utilisateur doit être **validé** (`0 <= k < N`) avant de calculer `&tab[k]`.

Le guide ANSSI demande d'initialiser les pointeurs, de les remettre à `NULL` après libération et de vérifier toute valeur de retour pouvant valoir `NULL`.

### EX 8 Le verrou const

★★ 5 min

`const` empêche de modifier ce qui est protégé. Pour chacune des déclarations suivantes, indiquez ce qui est autorisé (`*p = 5;` ? `p++;` ?), puis vérifiez avec gcc.

```c
int x = 1, y = 2;
const int *p1 = &x;        // pointeur vers un int constant
int *const p2 = &x;        // pointeur constant vers un int
const int *const p3 = &x;  // les deux
```

Pourquoi `compterOccurrences` et `maStrlen` prennent-elles un paramètre `const` ? Que se passe-t-il si `maStrlen` essaie d'écrire dans la chaîne ?

**INDICE**

Lisez la déclaration **de droite à gauche** : `int *const p2` se lit « p2 est un pointeur constant vers un int ».

**Fragment 4 :** `FLAG{` + le nom de l'option de gcc qui a détecté le piège C, sans le `-f` + `}`.

# Mission 4 — Le standard : pointeurs de fonctions

## Pourquoi pointer vers une fonction ?

Jusqu'ici, vos pointeurs désignaient des **données** (`int`, `char`, tableaux). Mais en C, les **fonctions** sont elles aussi stockées en mémoire, dans le segment de code, à une certaine adresse. On peut donc créer des pointeurs vers des fonctions. C'est utile pour :

- écrire des programmes **génériques** et **modulaires** (une fonction de tri qui ne connaît pas l'ordre de tri) ;
- passer une fonction en paramètre d'une autre (**fonction de rappel**, *callback*) ;
- construire des **tableaux de fonctions** (menus, interpréteurs de commandes, gestionnaires d'événements).

## Syntaxe

```c
type_retour (*nomPointeur)(liste_des_types_des_parametres);
```

```c
int addition(int a, int b)
{
    return a + b;
}

int (*ptrFonction)(int, int);      // pointeur vers une fonction (int, int) -> int
ptrFonction = addition;            // ou &addition (équivalent)
int resultat = ptrFonction(5, 3);  // appel indirect : resultat vaut 8
// ou : resultat = (*ptrFonction)(5, 3);   (équivalent)
```

> **PIÈGE CLASSIQUE** **Attention aux parenthèses !**
>
> `int (*ptr)(int, int);` déclare un **pointeur vers une fonction** qui renvoie un `int`. `int *ptr(int, int);` déclare une **fonction** nommée `ptr` qui renvoie un pointeur vers un `int`. Rien à voir !

## typedef pour simplifier

Pour rendre le code lisible, on donne un nom au type avec `typedef` :

operations.c

```c
#include <stdio.h>

typedef int (*OperationMath)(int, int);   // OperationMath = pointeur vers (int,int) -> int

int addition(int a, int b)       { return a + b; }
int soustraction(int a, int b)   { return a - b; }
int multiplication(int a, int b) { return a * b; }

int main(void)
{
    OperationMath op = addition;
    printf("5 + 3 = %d\n", op(5, 3));
    op = multiplication;
    printf("5 * 3 = %d\n", op(5, 3));
    return 0;
}
```

## Tableau de pointeurs de fonctions

On peut ranger des fonctions dans un tableau, très pratique pour un menu :

commandes.c

```c
#include <stdio.h>

typedef void (*Commande)(void);

void cmdNouveau(void)  { printf("Nouveau fichier créé\n"); }
void cmdOuvrir(void)   { printf("Fichier ouvert\n"); }
void cmdSauver(void)   { printf("Fichier sauvegardé\n"); }
void cmdQuitter(void)  { printf("Au revoir !\n"); }

int main(void)
{
    const Commande menu[] = {cmdNouveau, cmdOuvrir, cmdSauver, cmdQuitter};
    const char *libelles[] = {"Nouveau", "Ouvrir", "Sauvegarder", "Quitter"};
    const int nbCommandes = (int)(sizeof menu / sizeof menu[0]);
    int choix;

    printf("Menu : 0-Nouveau, 1-Ouvrir, 2-Sauvegarder, 3-Quitter\nVotre choix : ");
    if (scanf("%d", &choix) == 1 && choix >= 0 && choix < nbCommandes) {
        printf("Exécution de '%s'...\n", libelles[choix]);
        menu[choix]();                // appel de la fonction choisie
    } else {
        printf("Choix invalide\n");
    }
    return 0;
}
```

## EX 9 — Premiers pas avec les pointeurs de fonctions

★ 10 min

Partez de `calculatrice_depart.c`.

1. Créez quatre fonctions mathématiques simples : `int addition(int a, int b);`, `int soustraction(int a, int b);`, `int multiplication(int a, int b);`, `int division(int a, int b);`.
2. Dans le `main`, déclarez un pointeur de fonction `int (*operation)(int, int);` et testez-le successivement avec chaque opération : `operation = addition; printf("5 + 3 = %d\n", operation(5, 3));`, etc.
3. Créez un tableau de pointeurs contenant ces quatre fonctions, ainsi qu'un tableau de chaînes pour les symboles (`"+"`, `"-"`, `"*"`, `"/"`). Écrivez une boucle qui teste toutes les opérations.
4. Affichez avec `%p` l'adresse de `addition`, de `main` et d'une variable locale. Que constatez-vous ? (Pour afficher l'adresse d'une fonction, un cast `(void *)` est accepté par gcc mais n'est pas garanti par la norme : c'est une extension POSIX.)

## EX 10 — La calculatrice du standard

★★ 20 min

Créez une calculatrice interactive qui :

1. affiche un menu avec les 4 opérations ;
2. demande à l'utilisateur de choisir une opération ;
3. demande deux nombres ;
4. appelle la fonction correspondante **via le tableau de pointeurs** ;
5. affiche le résultat ;
6. recommence jusqu'à ce que l'utilisateur quitte (choix spécial, par exemple 5).

**INDICE**

Une boucle `while` et un choix spécial pour quitter suffisent. Le tableau de pointeurs remplace avantageusement un `switch` : ajouter une opération (modulo, puissance) revient à ajouter une fonction et une case au tableau.

**SÉCURITÉ** **Le menu qui saute n'importe où**

`operations[choix]()` lit une adresse dans le tableau, puis **saute** à cette adresse. Si l'utilisateur tape `7` ou `-3`, le programme lit une case hors du tableau et saute à une adresse arbitraire : au mieux un plantage, au pire l'exécution d'un code choisi par un attaquant (c'est le principe des détournements de flot d'exécution). **Validez** `choix` (bornes **et** retour de `scanf`) **avant** l'appel indirect. Testez aussi :

- `7 / 0` : la division par zéro tue le processus (signal `SIGFPE`) ;
- `-2147483648 / -1` (`INT_MIN / -1`) : le résultat ne tient pas dans un `int`, c'est un comportement indéfini qui plante aussi sur x86 ;
- `2147483647 + 1` : dépassement de capacité (vérifiez avec `-fsanitize=undefined`).

Proposez une signature qui permet à chaque opération de **signaler** une erreur au lieu de planter, par exemple `bool (*)(int a, int b, int *resultat)`.

**Fragment 5 :** `FLAG{` + le nom du signal reçu lors d'une division entière par zéro + `}`.

**POUR LES RAPIDES** **Fonctions de rappel et qsort** ★★★

1. Écrivez `void appliquer(int *tab, size_t n, int (*f)(int))` qui remplace chaque élément par `f(element)`. Testez avec une fonction `carre` et une fonction `oppose`.
2. La bibliothèque standard fournit un tri générique : `void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));` (`man 3 qsort`). Triez un tableau d'`int` dans l'ordre croissant, puis décroissant, en écrivant deux fonctions de comparaison.
3. Piège : pourquoi la comparaison `return *(const int *)a - *(const int *)b;` est-elle fausse pour certains tableaux ? Testez avec `INT_MAX` et `INT_MIN`, et proposez une version sans dépassement.
4. Teaser du sujet 04 : avec un tableau de pointeurs vers des fonctions de tri (bulles, sélection, insertion), vous construirez un **système de tri modulaire** qui compare les algorithmes… sur des tableaux alloués dynamiquement.

# Bilan de la séance

**À RETENIR**

- `&x` est l'adresse de `x` ; si `p = &x`, alors `*p` **est** `x`.
- `p1 = p2` copie une adresse ; `*p1 = *p2` copie une valeur.
- `p + i` avance de `i` **éléments** ; `p - q` compte des éléments (`ptrdiff_t`, `%td`).
- On peut former l'adresse « un après » la fin d'un tableau, jamais la déréférencer ; on ne forme jamais d'adresse avant le début.
- Un pointeur s'initialise toujours (au besoin à `NULL`), se teste avant usage, et ne pointe jamais vers une locale disparue.
- Une adresse s'affiche avec `%p` et un cast `(void *)`.
- Un pointeur de fonction se déclare `type (*nom)(params)` ; on valide l'indice **avant** tout appel via un tableau de fonctions.

- ☐ J'ai mes 5 fragments de clé dans `badge03.txt`.
- ☐ `pieges.c` corrigé ne déclenche plus aucun rapport d'ASan.

☐ Ma calculatrice résiste à `7`, `-3`, `abc`, `7 / 0` et `INT_MIN / -1`.

# Aller plus loin

## Ressources

| Ressource | Pour quoi faire |
|---|---|
| `man 3 qsort`, `man 3 bsearch` | Les deux fonctions standard qui prennent un pointeur de fonction. |
| cppreference — Pointer declaration | Pointeurs, pointeurs de fonctions, `const`, règles de la norme. |
| cppreference — Pointer arithmetic | Ce qui est défini (et ce qui ne l'est pas) en arithmétique des pointeurs. |
| cdecl.org | Traduit en anglais une déclaration compliquée (`int (*(*f)(int))[10]`). |
| Kernighan & Ritchie, *Le langage C*, chapitre 5 | « Pointeurs et tableaux » : la référence historique. |
| M. Kerrisk, *The Linux Programming Interface*, chap. 6 | Organisation mémoire d'un processus (segments, pile, tas). |
| A. Tanenbaum, *Systèmes d'exploitation* | Mémoire virtuelle et espaces d'adressage. |
| J.-M. Rifflet, *La programmation sous UNIX* | Référence francophone pour le reste du module. |
| Guide ANSSI, règles de programmation sécurisée en C (dossier du CM) | Règles sur l'initialisation, la vérification et l'arithmétique des pointeurs. |
| `gdb` | `print p`, `print *p`, `print &i`, `x/4dw tab` (examine 4 entiers à partir de `tab`). |
| `valgrind ./prog` | Détecte lectures non initialisées et accès invalides (alternative à ASan). |

## Pour approfondir

> **CULTURE** **Tableau et pointeur : cousins, pas jumeaux**
>
> Dans la plupart des expressions, le nom d'un tableau est converti en pointeur vers son premier élément : `tab[i]` est **par définition** `*(tab + i)` (d'où la curiosité légale `i[tab]`). Mais un tableau n'est pas un pointeur : `sizeof tab` donne la taille de tout le tableau, et on ne peut pas écrire `tab = autre;`. Passé à une fonction, un tableau devient un simple pointeur et perd sa taille.

> **CULTURE** **Mémoire virtuelle**
>
> Les adresses que vous affichez sont **virtuelles** : chaque processus a son propre espace d'adressage, traduit en adresses physiques par le matériel (MMU) et le noyau. Deux processus peuvent donc afficher la même adresse pour deux données différentes. Vous le verrez au sujet 06 avec `fork`, et en détail dans le cours sur la mémoire virtuelle. Pour voir la carte mémoire d'un processus : `cat /proc/<pid>/maps`.

**CULTURE** **Pointeurs de fonctions dans le système**

Les pointeurs de fonctions sont partout dans la programmation système : `signal(SIGINT, gestionnaire)` (gestionnaire de signal), `pthread_create(&t, NULL, fonction, arg)` (sujet 08), `atexit(f)`, ou encore les tables d'opérations des pilotes du noyau Linux (`struct file_operations`). Ils sont aussi une cible de choix pour les attaquants, d'où des protections comme le *Control-Flow Integrity*.